

INTELLIGENT AGENTS

The emergence of intelligent agents (1995–present)

Perhaps encouraged by the progress in solving the subproblems of AI, researchers have also started to look at the “whole agent” problem again. The work of Allen Newell, John Laird, and Paul Rosenbloom on SOAR (Newell, 1990; Laird et al., 1987) is the best-known example of a complete agent architecture.

One of the most important environments for intelligent agents is the Internet. AI systems have become so common in Web-based applications that the “-bot” suffix has entered everyday language. Moreover, AI technologies underlie many Internet tools, such as search engines, recommender systems, and Web site aggregators.

One consequence of trying to build complete agents is the realization that the previously isolated subfields of AI might need to be reorganized somewhat when their results are to be tied together.

In particular, it is now widely appreciated that sensory systems (vision, sonar, speech recognition, etc.) cannot deliver perfectly reliable information about the environment. Hence, reasoning and planning systems must be able to handle uncertainty.

A second major consequence of the agent perspective is that AI has been drawn into much closer contact with other fields, such as control theory and economics, that also deal with agents.

Recent progress in the control of robotic cars has derived from a mixture of approaches ranging from better sensors, control-theoretic integration of sensing, localization and mapping, as well as a degree of high-level planning.

Here we will use the concept of rationality that is applied to a wide variety of agents operating in any imaginable environment to develop a small set of design principles for building successful agents—systems that can reasonably be called intelligent.

THE CONCEPT OF RATIONALITY

A rational agent is one that does the right thing—conceptually speaking, every entry in the table for the agent function is filled out correctly. Obviously, doing the right thing is better than doing the wrong thing, but what does it mean to do the right thing?

When an agent is plunked down in an environment, it generates a sequence of actions according to the percepts it receives. This sequence of actions causes the environment to go through a sequence of states. If the sequence is desirable, then the agent has performed well. This notion of desirability is captured by a performance measure that PERFORMANCE MEASURE evaluates any given sequence of environment states.

What is rational at any given time depends on four things:

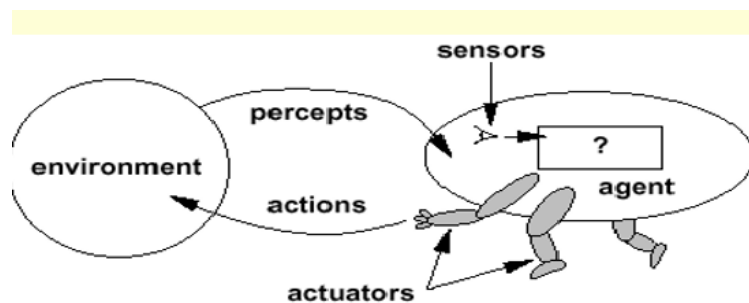
- The performance measure that defines the criterion of success.
- The agent's prior knowledge of the environment.
- The actions that the agent can perform.
- The agent's percept sequence to date.

What is an agent?

An **agent** is anything that can be viewed as **perceiving** its environment through **sensors** and **acting** upon that environment through **actuators** (effectors).

A human agent has eyes, ears, and other organs for sensors, and hands, legs, mouth, and other body parts for effectors.

A robotic agent substitutes cameras and infrared range finders for the sensors and various motors for the effectors.



Example:

Human agent: Eyes, ears, and other organs for sensors; hands, legs, mouth, and other body parts for actuators

Robotic agent: Cameras and infrared range finders for sensors, various motors for actuators.

Percept

- Agent's perceptual inputs at any given instant

Percept sequence

- A complete history of everything that the agent has ever perceived

Agents behaviour

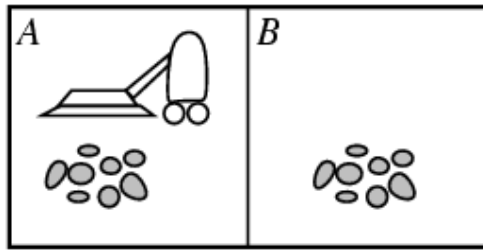
Mathematically speaking, we say that an agent's behavior is described by the agent function that maps any given percept sequence to an action.

$$[f: P^* \rightarrow A]$$

The agent program runs on the physical architecture to produce f
agent = architecture + program

EXAMPLE:

Vacuum-cleaner world



Percepts: location and contents, e.g., [A,Dirty]

Actions: *Left*, *Right*, *Suck*, *NoOp*

Agent's function → look-up table

Look-up table

Percept sequence	Action
[A, <i>Clean</i>]	<i>Right</i>
[A, <i>Dirty</i>]	<i>Suck</i>
[B, <i>Clean</i>]	<i>Left</i>
[B, <i>Dirty</i>]	<i>Suck</i>
[A, <i>Clean</i>], [A, <i>Clean</i>]	<i>Right</i>
[A, <i>Clean</i>], [A, <i>Dirty</i>]	<i>Suck</i>
⋮	⋮

Learning

Does a rational agent depend on only current percept?

No, the past percept sequence should also be used. This is called **learning**

After experiencing an episode, the agent should adjust its behaviors to perform better for the same job next time.

Autonomy

If an agent just relies on the prior knowledge of its designer rather than its own percepts, then the agent lacks **autonomy**

A rational agent should be autonomous- it should learn what it can to compensate for partial or incorrect prior knowledge.

E.g., a clock

- No input (percepts)
- Runs but only on its own algorithm (prior knowledge)
- No learning, no experience, etc.

Software Agents

Sometimes, the environment may not be the real world E.g., flight simulator, video games, Internet. They are all artificial but very complex environments. Those agents working in these environments are called software agent (softbots). Because all parts of the agent are software

Task environments

Task environments are the problems, while the rational agents are the solutions

Specifying the task environment

PEAS description as fully as possible

- Performance
- Environment
- Actuators
- Sensors

In designing an agent, the first step must always be to specify the task environment as fully as possible.

Example: Automated taxi drive.

Performance measure

How can we judge the automated driver? Which factors are considered?

- getting to the correct destination.
- minimizing fuel consumption.
- minimizing the trip time and/or cost.
- minimizing the violations of traffic laws.
- maximizing the safety and comfort, etc.

Environment

A taxi must deal with a variety of roads, Traffic lights, other vehicles, pedestrians, stray animals, road works, police cars, etc. Interact with the customer.

Actuators (for outputs)

Control over the accelerator, steering, gear shifting and braking. Must have a display to communicate with the customers

Sensors (for inputs)

Detect other vehicles, road situations, GPS (Global Positioning System) to know where the taxi is, Many more other devices are necessary.

Environment types

Fully observable (vs. partially observable)

Deterministic (vs. stochastic)

Episodic (vs. sequential)

Static (vs. dynamic)

Discrete (vs. continuous)

Single agent (vs. multiagent):

Properties of task environments

Fully observable vs. Partially observable

Is everything an agent requires to choose its actions available to it via its sensors?

Perfect or Full information.

If so, the environment is fully accessible. If not, parts of the environment are inaccessible

Agent must make informed guesses about world.

In decision theory: perfect information vs. imperfect information.

If an agent's sensors give it access to the complete state of the environment at each point in time, then the environment is effectively and fully observable.

if the sensors detect all aspects that are relevant to the choice of action

Partially observable

An environment might be Partially observable because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data.

Example:

A local dirt sensor of the cleaner cannot tell whether other squares are clean or not

Deterministic vs. stochastic

Does the change in world state depend only on current state and agent's action? Or if the next state of the environment completely determined by the current state and the actions executed by the agent, then the environment is deterministic, otherwise, it is Stochastic

Non-deterministic environments

Have aspects beyond the control of the agent

Utility functions have to guess at changes in world

Example

Strategic environment: deterministic except for actions of other agents

Cleaner and taxi driver are stochastic because of some unobservable aspects à noise or unknown.

Episodic vs. sequential

Is the choice of current action dependent on previous actions?

If not, then the environment is episodic

In non-episodic environments, agent has to plan ahead. The current choice will affect future actions

An episode = agent's single pair of perception & action

The quality of the agent's action does not depend on other episodes. Every episode is independent of each other

Episodic environment is simpler. The agent does not need to think ahead

Sequential

- Current action may affect all future decisions

-Ex. Taxi driving and chess.

Static vs. dynamic

Static environments don't change while the agent is deliberating over what to do. E.g., the destination

Dynamic environments do change, so agent should/could consult the world when choosing actions. Alternatively: anticipate the change during deliberation OR make decision very fast. A dynamic environment is always changing over time. E.g., the number of people in the street

Semi-dynamic: If the environment itself does not change with the passage of time but the agent's performance score does.

Discrete vs. continuous

A limited number of distinct, clearly defined percepts and actions vs. a range of values (continuous)

If there are a limited number of distinct states, clearly defined percepts and actions, the environment is discrete.

E.g., Chess game

Continuous: Taxi driving

Single agent VS. multiagent

An agent operating by itself in an environment or there are many agents working together

Playing a crossword puzzle – single agent

Chess playing – two agents

Competitive multiagent environment

Chess playing

Cooperative multiagent environment

Automated taxi driver

Avoiding collision

Known vs. unknown

This distinction refers not to the environment itself but to the agent's (or designer's) state of knowledge about the environment.

-In known environment, the outcomes for all actions are given. (example: solitaire card games).

- If the environment is unknown, the agent will have to learn how it works in order to make good decisions.(example: new video game).

Structure of agents

Agent = architecture + program

Architecture = some sort of computing device (sensors + actuators)

(Agent) Program = some function that implements the agent mapping = “?”

Agent Program = Job of AI

Agent programs

An agent program takes current percept as input from the sensors and return an action to the actuators

A simple agent program can be defined mathematically as an agent function which maps every possible percepts sequence to a possible action the agent can perform.

Input for Agent Program

- Only the current percept

Input for Agent Function

- The entire percept sequence (entire percept history, that is, the agent must remember all of them)

The agent program can Implement as a look up table (agent function)

Skeleton design of an agent program

```
function TABLE-DRIVEN-AGENT(percept) returns action
  static: percepts, a sequence, initially empty
         table, a table, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

P = the set of possible percepts

T= lifetime of the agent (The total number of percepts it receives)

$$\sum_{t=1}^T |P|^t$$

Consider the automated taxi: the visual input from a single camera comes in at the rate of roughly 27 megabytes per second (30 frames per second, 640×480 pixels with 24 bits of color information). This gives a lookup table with over $10^{250,000,000,000}$ entries for an hour's driving.

Even the lookup table for chess—a tiny, well-behaved fragment of the real world—would have at least 10^{150} entries.

The daunting size of these tables (the number of atoms in the observable universe is less than 10^{80}) means that:

- (a) no physical agent in this universe will have the space to store the table,
- (b) the designer would not have time to create the table, (c) no agent could ever learn all the right table entries from its experience, and
- (d) even if the environment is simple enough to yield a feasible table size, the designer still has no guidance about how to fill in the table entries.

Types of agent programs (4 types)

- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents

Simple reflex agents

It is the simplest kind of an agent. These agents select actions on the basis of the current percept, ignoring the rest of the percept history.

It uses just ***condition-action rules***

The rules are like the form “if ... then ...”

if car-in-front-is-braking then initiate-braking.

They are efficient, but have narrow range of applicability because knowledge sometimes cannot be stated explicitly.

It works only if the environment is fully observable.

For example, the vacuum agent whose agent function is tabulated as above, is a simple reflex agent, because its decision is based only on the current location and on whether that location contains dirt.

```

function SIMPLE-REFLEX-AGENT(percept) returns action
  static: rules, a set of condition-action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← RULE-ACTION[rule]
  return action

```

Model-based Reflex Agents

For the world that is partially observable the agent has to keep track of an internal state that depends on the percept history reflecting some of the unobserved aspects.

E.g., driving a car and changing lane requiring two types of knowledge:

- How the world evolves independently of the agent
- How the agent's actions affect the world

The most effective way to handle partial observability is for the agent to *keep track of the part of the world it can't see now*. That is, the agent should maintain some sort of **internal state** that depends on the percept history and thereby reflects at least some of the unobserved aspects of the current state.

E.g.

For the braking problem, the internal state is not too extensive—just the previous frame from the camera, allowing the agent to detect when two red lights at the edge of the vehicle go on or off simultaneously

Goal-based agents

Current state of the environment is always not enough. The goal is another issue to achieve judgment of rationality / correctness.

Knowing something about the current state of the environment is not always enough to decide what to do. For example, at a road junction, the taxi can turn left, turn right, or go straight on. The correct decision depends on where the taxi is trying to get to. In other words, as well as a current state description, the GOAL agent needs some sort of goal information that describes situations that are desirable—for example, being at the passenger's destination. The agent program can combine this with the model (the same information as was used in the model-based reflex agent) to choose actions that achieve the goal.

Utility-based agents

Goals alone are not enough to generate **high-quality** behavior

E.g. meals in Canteen, good or not?

Many action sequences the goals some are better and some worse

if goal means success, then **utility** means the degree of success (how successful it is)

it is said state A has higher utility, If state A is more preferred than others, Utility is therefore a function that maps a state onto a real number-the degree of success.

Goals alone are not enough to generate high-quality behavior in most environments. For example, many action sequences will get the taxi to its destination (thereby achieving the goal) but some are quicker, safer, more reliable, or cheaper than others. Goals just provide a crude binary distinction between “happy” and “unhappy” states.

A more general performance measure should allow a comparison of different world states according to exactly how happy they would make the agent. Because “happy” does not sound very scientific, economists and computer scientists use the term utility instead.

Utility has several advantages:

- When there are conflicting goals, only some of the goals but not all can be achieved. utility describes the appropriate trade-off
- When there are several goals. None of them are achieved **certainly**, utility provides a way for the decision-making

Learning Agents

After an agent is programmed, can it work immediately?

No, it still need teaching. In AI, once an agent is done, we teach it by giving